

Fluxo de Trabalho em Grupo

Fluxo de Trabalho em Grupo — Git & GitHub

FASE 0 — Antes de começar (só quem vai criar o repositório)

Uma pessoa do grupo cria o repositório no GitHub: 1. Site do GitHub → **New repository** 2. Dá um nome (ex: `trabalho-gastos-mensais`) 3. **Deixa tudo desmarcado** (sem README, sem `.gitignore`) — evita conflito de histórico 4. Cria o repositório 5. Vai em **Settings** → **Collaborators** e adiciona os outros integrantes pelo usuário do GitHub

FASE 1 — Configuração (cada integrante faz isso no PRÓPRIO computador, uma única vez)

```
git config --global user.name "Seu Nome"
git config --global user.email "seuemail@exemplo.com"
git config --global init.defaultBranch main
```

Se ainda não tiver chave SSH configurada nesse computador:

```
ssh-keygen -t ed25519 -C "seuemail@exemplo.com"
cat ~/.ssh/id_ed25519.pub
```

Copia o resultado e cadastra em: GitHub → foto de perfil → **Settings** → **SSH and GPG keys** → **New SSH key**

Testa se funcionou:

```
ssh -T git@github.com
```

FASE 2 — Entrando no projeto (cada integrante faz isso uma vez)

Em vez de `git init`, como o repositório **já existe** no GitHub, usa `clone`:

```
git clone git@github.com:usuario/trabalho-gastos-mensais.git
cd trabalho-gastos-mensais
```

Isso já baixa tudo — não precisa configurar `remote add origin`, o clone já faz isso.

FASE 3 — Cada um cria sua própria branch (linha do tempo individual)

```
git checkout -b gastos-seunome
```

Exemplo:

```
git checkout -b gastos-andre
```

Por quê: assim cada pessoa mexe na sua própria “área”, sem correr risco de sobrescrever o trabalho de outro integrante enquanto todos estão editando ao mesmo tempo.

FASE 4 — Trabalhando (cada um faz isso quantas vezes precisar)

Crie seu arquivo de gastos, por exemplo:

```
touch gastos_andre.csv
```

Depois de editar/preencher o arquivo:

```
git status
```

(sempre bom conferir o que vai ser salvo antes de continuar)

```
git add gastos_andre.csv
git commit -m "Adiciona gastos do mês de julho"
```

Envie sua branch pro GitHub (funciona como um backup, e os outros já conseguem ver seu progresso):

```
git push -u origin gastos-andre
```

(depois da primeira vez, só precisa de `git push`)

Repita `add` → `commit` → `push` sempre que atualizar algo.

FASE 5 — Juntando tudo no final (uma pessoa faz isso, ou todos acompanhando)

1. Vá para a branch principal e atualize ela

```
git checkout main
git pull
```

2. Busque as branches mais recentes de todo mundo

```
git fetch origin
```

3. Junte cada branch de integrante na main

```
git merge origin/gastos-andre
git merge origin/gastos-maria
git merge origin/gastos-joao
```

Se os arquivos tiverem **nomes diferentes** (`gastos_andre.csv` , `gastos_maria.csv` ...), isso deve rolar **sem nenhum conflito**.

4. Envie a main atualizada, já com tudo junto

```
git push
```

FASE 6 — Se aparecer conflito de merge

Vai aparecer algo assim no terminal:

```
CONFLICT (content): Merge conflict in algum_arquivo.csv
```

Abra o arquivo, você vai ver marcações assim:

```
<<<<<< HEAD
sua versão
=====
versão da outra pessoa
>>>>>> nome-da-branch
```

Edite manualmente, decida o que fica, apague as marcações (`<<<<<<` , `=====` , `>>>>>>`), depois:

```
git add nome-do-arquivo
git commit -m "Resolve conflito entre branches"
git push
```

Regra de ouro pra evitar dor de cabeça amanhã

✓ Cada pessoa cria um **arquivo com nome próprio** — evita 90% dos conflitos possíveis ✓ Antes de mexer em qualquer coisa, sempre rode `git status` primeiro ✓ Committe com frequência, com mensagens curtas e claras ✓ Combine com o grupo: **quem vai ser responsável por fazer o merge final** na `main`

Resumo visual do fluxo completo

```
main (repositório criado)
|
├─ gastos-andre    → André trabalha, commita, dá push
├─ gastos-maria    → Maria trabalha, commita, dá push
├─ gastos-joao     → João trabalha, commita, dá push
|
▼ (no final, alguém junta tudo)
main ← merge de todas as branches
|
▼
git push  (versão final no GitHub)
```

Se o Vim abrir pedindo mensagem (durante merge)

1. `Esc`
2. Digite `:wq`
3. `Enter`